HOLLOWED RUINS

Haunted-maze survival game built in Unity 6 for `CMPS434 - Game Development Course Project`.

> *A shifting fortress. A patient Warden. A broken seal that is the only way out.*

Contents

- Game Snapshot
- Player Manual
- Story
- Gameplay Overview
- Level Profiles
- Project Overview
- Getting Started
- Project Structure
- Gameplay Systems Reference
- Assets And Content
- Scene Setup Expectations
- Production Notes
- Credits

Game Snapshot

Item	Details
Genre	Third-person horror exploration / tactical puzzle
Core Hook	Escape a procedural maze while surviving ghost-triggered 4x4 chess duels
Engine	`Unity 6000.3.9f1`
Platform	Windows 10/11
Input	Keyboard + mouse, with Input System gamepad bindings present
Primary Goal	Collect all key fragments, reveal the portal, and escape
Failure State	Lose all 3 hearts by failing chess duels

Player Manual

Game Premise

You are trapped inside a haunted ruin maze. Explore the labyrinth, collect every broken key piece, reveal the exit portal, and escape before the Warden catches you.

If the Warden reaches you, the game shifts into a 4x4 chess duel. Winning the duel buys time and forces him away. Losing the duel costs a heart. Lose all hearts and the run ends.

Controls

Input	Action
`W / A / S / D`	Move
`Mouse`	Look around
`Left Shift`	Sprint
`Left Click`	Select and move chess pieces during duels
`Esc`	Pause / open in-game menu
`R`	Restart the current level
`Q`	Dash, final-heart mode only
`Space`	Jump, final-heart mode only

Objective Flow

1. Search the maze for key fragments.
2. Avoid or outlast the Warden.
3. Win chess duels when caught.
4. Assemble the full seal.
5. Reach the portal and escape.

Win / Lose Conditions

Result	Condition
`Win`	Collect all key pieces and enter the exit portal
`Lose`	Lose all 3 hearts by failing chess duels

HUD

HUD Element	Purpose
`Hearts`	Top-left life display
`Key Counter`	Shows progress such as `2 / 3`
`Portal Prompt`	Changes to `Find the portal!` after the final key is collected
`Ghost Warning`	Pulses when the Warden is actively chasing
`Minimap`	Top-right tactical view with a cyan player marker
`Last Heart Overlay`	Red heartbeat screen effect on the final life
`Dash Indicator`	Displays emergency dash cooldown

Story

The Legend

Deep beneath the hills of Ashen Hollow lies a fortress no map remembers. It was built centuries ago by Lord Aldric Voss, a chess grandmaster, occultist, and tyrant who ruled through fear and ritual rather than open war.

Aldric believed the mind was the truest prison. To prove it, he shaped his fortress into a shifting labyrinth, a place where corridors changed, hope decayed, and escape became a psychological game. He did not simply torment prisoners. He studied them. He sat them across a chessboard in the dark and offered one cruel bargain: win and live another day, lose and surrender something of yourself. A memory. A year. A piece of the soul.

No one ever defeated him.

When Aldric died, he did not leave his fortress behind. His madness fused with the stone, the shadows, and the labyrinth itself. What remains is the Warden, a restless presence bound to the ruins and still searching for new prisoners to test.

You

You are an urban explorer who followed the rumors into Ashen Hollow. Buried beneath local folklore was talk of a forgotten fortress, a shattered relic, and a treasure no one ever brought back out.

You entered looking for history.

Now you are looking for the exit.

The walls shift behind you. Torchlight dies corridor by corridor. Somewhere in the dark, something old and patient is already tracking your footsteps.

The Warden has found a new prisoner.

The Chess Duels

When the Warden catches you, he does not kill you. That was never his way.

Instead, he drags you into the ritual that defined his rule. A board forms from shadow and memory, and the hunt becomes a duel of control. Every encounter is a fragment of the lessons he once forced on his captives: survive, protect, sacrifice, endure.

Winning breaks his focus for a moment and gives you room to run.

Losing costs part of what keeps you grounded in the world above. In gameplay terms, that loss becomes one heart. Three failures, and the ruins claim you like everyone before you.

The Key And The Portal

The relic hidden throughout the maze is the Warden's Seal, the only object capable of opening the passage out. Aldric shattered it himself, convinced that no prisoner could ever reassemble it while being hunted inside his labyrinth.

Your escape depends on doing exactly that:

- Find every broken key piece
- Restore the seal
- Open the portal
- Run before the Warden reaches you again

Some doors were built to keep things out.

This one was built to keep you in.

Gameplay Overview

The Warden

- Patrols the maze using Unity NavMesh
- Detects the player through line of sight
- Investigates sound events emitted by movement and pickups
- Responds more aggressively to sprinting than walking
- Is strongly drawn to key pickups because they emit a large noise pulse

Chess Duel Rules

When the Warden catches the player, exploration pauses and a chess challenge begins on a 4x4 board.

Possible duel objectives:

- Survive for a number of turns
- Protect a specified piece for a number of turns
- Avoid losing any white piece for a number of turns
- Capture a specified enemy piece before turns run out

Possible duel outcomes:

- `Win` : the Warden screams, vanishes temporarily, and respawns elsewhere in the maze
- `Lose` : the player loses 1 heart, the Warden is briefly stunned, and exploration resumes

Final Heart Mode

When only one heart remains:

- `Q` activates a forward dash with a 30-second cooldown
- `Space` enables jumping
- Sprinting gains a speed multiplier
- A heartbeat overlay appears across the HUD

Level Profiles

Each playable scene changes the pressure curve through maze size, collectible count, and ghost speed.

Level	Maze	Keys	Patrol Speed	Chase Speed
Tutorial	7x7	2	3	6
Easy	12x12	3	4	8
Medium	15x15	4	5	9
Hard	17x17	5	6	10

Project Overview

High Concept

`Hollowed Ruins` combines third-person maze exploration with short-form tactical chess encounters. The main tension comes from alternating between spatial survival and turn-based problem solving.

Core Loop

1. Spawn into a procedurally generated maze.
2. Explore, collect key pieces, and manage ghost pressure.
3. Enter a chess duel if the Warden catches the player.
4. Resume exploration after duel resolution.
5. Escape through the revealed portal.

Build Scenes

Configured build scenes from `ProjectSettings/EditorBuildSettings.asset` :

- `Assets/_Scenes/MainMenu.unity`

- `Assets/_Scenes/LevelTutorial.unity`
- `Assets/_Scenes/LevelEasy.unity`
- `Assets/_Scenes/LevelMedium.unity`
- `Assets/_Scenes/LevelHard.unity`

Getting Started

Opening The Project

1. Open Unity Hub.
2. Add the folder `Hollowed Ruins`.
3. Open the project with `Unity 6000.3.9f1`.
4. Load `Assets/_Scenes/MainMenu.unity` to start from the intended front end.

Primary Packages

- `com.unity.inputsystem`
- `com.unity.cinemachine`
- `com.unity.ai.navigation`
- `com.unity.render-pipelines.universal`
- `com.unity.ugui`
- `com.unity.textmeshpro`

Project Structure

High-Value Folders

Path	Purpose
`Hollowed Ruins/Assets/_Scenes`	Main menu and gameplay scenes
`Hollowed Ruins/Assets/_Scenes/Levels Config`	Per-difficulty `LevelConfig` assets
`Hollowed Ruins/Assets/Scripts`	Gameplay code
`Hollowed Ruins/Assets/Audio`	Ambient, music, and ghost SFX
`Hollowed Ruins/Assets/Art`	Character art, materials, textures, and animation controllers
`Hollowed Ruins/Assets/UI`	HUD sprites and fonts
`Hollowed Ruins/Assets/DungeonModularPack`	Modular environment kit used by the maze builder
`docs`	Design and script reference documents

Runtime Code Layout

Folder	Responsibility
`Scripts/Core`	Game state, health, noise, level tuning
`Scripts/Player`	Player movement and spawn support
`Scripts/Maze`	Procedural generation, objective placement, exit flow
`Scripts/Ghost`	AI, animation bridge, spawn support
`Scripts/Chess`	Duel rules, board model, scenarios, evaluator, AI
`Scripts/UI`	HUD, minimap, menus, duel board, end screens

Gameplay Systems Reference

This section is written as a production handoff: what each system owns, what it depends on, and why it matters to the game.

1. State Management

`GameStateManager`

Central authority for game flow. All major systems query or react to `Exploring`, `ChessDuel`, `GameOver`, and `Win`.

Key responsibilities:

- Stores the active `LevelConfig`
- Broadcasts state changes through `OnStateChanged`
- Freezes and resumes world time with `Time.timeScale`
- Persists across scene reloads with `DontDestroyOnLoad`

`HealthSystem`

Owns the player's heart economy.

Key responsibilities:

- Initializes and resets hearts
- Emits `OnHeartsChanged`
- Triggers `GameOver` when hearts reach zero

`NoiseSystem`

Simple event bus for sound-driven gameplay.

Key responsibilities:

- Emits noise events with `position` and `radius`
- Lets the player and pickup systems influence ghost behavior without hard references

`LevelConfig`

ScriptableObject that defines difficulty tuning per level.

Fields:

- `mazeWidth`
- `mazeHeight`
- `keyPieceCount`
- `ghostPatrolSpeed`
- `ghostChaseSpeed`

2. Player Layer

`PlayerController`

Primary exploration controller.

Required components:

- `CharacterController`
- `PlayerInput`

Serialized dependencies:

- `cameraTarget`
- `animator`
- `footstepSource`

Major features:

- Walk and sprint locomotion
- Mouse-look orbit target control
- Gravity handling through `CharacterController`
- Noise emission tied to movement state
- Footstep loop playback
- Last-heart sprint boost
- Last-heart dash and jump

Notable implementation details:

- `DashCooldownRemaining` and `DashReady` expose player status cleanly to HUD systems
- `EmitLoudNoise(float radius)` provides an explicit hook for future interactables
- Debug keys currently exist in `Update()`: `P` forces a chess duel and `L` removes a heart

`PlayerSpawner`

Legacy utility for spawning a prefab at a `PlayerSpawn` marker.

3. Maze Generation And Objective Layer

`MazeGenerator`

Builds the maze at runtime and places mission-critical actors and objectives.

Required dependencies:

- `NavMeshSurface` on the same GameObject for runtime bake
- Assigned `floorPrefab`, `wallPrefab`, `keyPiecePrefab`, and `exitPrefab`
- References to the player transform and ghost actor

Major features:

- Reads width, height, and key count from `GameStateManager.levelConfig`
- Uses recursive backtracking to carve a solvable maze
- Forces a central 2x2 hall for reliable spawn space
- Adds extra gaps to create loops and reduce single-solution predictability
- Builds floor, ceiling, and wall geometry
- Bakes a runtime NavMesh for ghost navigation
- Places the player, ghost, exit, and key pieces
- Notifies `MinimapFog` once the layout is ready

`PieceCollectionSystem`

Tracks objective progress.

Responsibilities:

- Counts collected key pieces
- Broadcasts progress to UI
- Fires completion events
- Reveals the exit once all keys are found

`KeyPiece`

Collectible actor attached to the key prefab.

Behavior:

- Applies bobbing and spinning for visual readability
- Detects player trigger entry
- Emits a very large noise pulse on pickup
- Reports progress to `PieceCollectionSystem`
- Destroys itself after collection

`ExitTrigger`

Win-condition gate attached to the exit portal.

Behavior:

- Verifies all key pieces are collected
- Pushes game state to `Win`

4. Ghost AI Layer

`GhostAI`

Primary enemy behavior controller.

Required components:

- `NavMeshAgent`

Serialized dependencies:

- `sightBlockMask`
- `audioSource`
- `ghostAlertClip`
- `ghostRoarClip`

States:

- `Patrol`
- `Chase`
- `Stun`

Major features:

- Reads movement tuning from `LevelConfig`
- Waits until runtime NavMesh is ready before activating navigation
- Uses forward-angle and raycast checks for visual detection
- Reacts to `NoiseSystem` events for investigation behavior
- Starts chess duels on collision with the player
- Supports `Stun()` and `Vanish(float duration)` as separate recovery outcomes

Important design note:

- Losing a duel stuns the ghost in place
- Winning a duel makes the ghost vanish, relocate, and then resume patrol

`GhostAnimator`

Bridges AI state into animation parameters.

Responsibilities:

- Pushes `IsMoving`
- Pushes `IsChasing`
- Pushes `IsStunned`
- Pushes `Speed`
- Triggers `Scream`

`GhostSpawner`

Additional spawn utility for `GhostSpawn` markers.

5. Chess Duel Layer

This subsystem is the identity feature of the project. It is cleanly split between orchestration, board rules, scenario data, objective evaluation, and AI decision-making.

`ChessDuelManager`

Top-level duel orchestrator.

Responsibilities:

- Starts duels when the game state changes to `ChessDuel`
- Loads a random scenario
- Builds the board model
- Handles player clicks and move submission
- Delays and executes ghost turns
- Evaluates win/loss outcome
- Plays duel music
- Resolves post-duel recovery back into exploration

Important runtime behavior:

- Uses `WaitForSecondsRealtime` so chess flow still runs while world time is paused
- Raises UI-facing events such as `OnDuelStarted`, `OnPlayerMoved`, and `OnTurnsRemainingChanged`

`ChessBoard`

Pure board-state model.

Responsibilities:

- Loads a scenario into a 4x4 grid
- Stores live piece positions
- Generates legal moves
- Executes captures
- Supports cloning for AI simulation

Supported piece logic:

- King
- Queen
- Rook
- Bishop
- Knight
- Pawn

`ChessData`

Shared enums and serializable data containers for the duel feature.

Includes:

- `PieceType`
- `PieceColor`
- `ObjectiveType`
- `ChessPiece`
- `PiecePlacement`
- `ChessObjective`

`ChessScenario`

Designer-authored ScriptableObject used to build an encounter.

Contains:

- Initial piece placements
- Objective metadata
- Designer-facing description text for UI

`ChessObjectiveEvaluator`

Evaluates duel success and failure conditions.

Supported objective types:

- `DontLosePiece`
- `ProtectPiece`
- `CaptureTarget`
- `SurviveNTurns`

`GhostChessAI`

Turn-selection logic for the ghost.

Implementation:

- Scores moves using a lightweight minimax search
- Evaluates board advantage by material value
- Plays as black

6. UI And Front-End Layer

`UIManager`

Activates the correct screen for each game state.

Panels:

- HUD
- Chess Duel
- Game Over
- Win

`HUDController`

Owns the live exploration HUD.

Dependencies:

- Heart icon images and sprites
- Piece counter `TextMeshProUGUI`
- Ghost warning icon
- Last-heart overlay image

- Dash indicator image and text

Responsibilities:

- Reflect heart changes
- Reflect key progress
- Switch prompt to `Find the portal!`
- Pulse ghost-warning alpha during chase
- Animate the heartbeat overlay
- Show dash readiness and cooldown

`ChessBoardUI`

Visual front end for the chess duel.

Dependencies:

- `GridLayoutGroup`
- Cell prefab using `ChessBoardCell`
- White and black piece sprite sets
- Objective and turns labels

Responsibilities:

- Builds the 4x4 board dynamically
- Renders pieces from the pure board model
- Highlights selection and legal moves
- Sends player clicks to `ChessDuelManager`

`ChessBoardCell`

Single reusable board-square prefab script.

Expected child references:

- background image
- highlight image
- piece image

Responsibilities:

- Initializes board coordinates
- Displays piece sprite and tint
- Forwards button clicks back to `ChessBoardUI`

`MinimapController`

Creates and updates the tactical minimap camera.

Responsibilities:

- Configures an orthographic top-down camera
- Positions it over the maze center
- Spawns a cyan cylinder marker for the player
- Hides minimap-only layers from the main camera

`MinimapFog`

Implements simple fog-of-war discovery for the minimap.

Responsibilities:

- Builds per-cell fog quads after the maze is generated
- Reveals the current cell and adjacent cells as the player moves

`MainMenuManager`

Controls the front-end flow from main panel to level select.

Responsibilities:

- Shows and hides menu panels
- Loads the tutorial, easy, medium, and hard scenes
- Quits the game or stops play mode in the editor

`GameScreensUI`

Owns restart and return-to-menu actions from end screens.

Responsibilities:

- Resets game state, hearts, and key progress
- Reloads the current scene
- Returns to `MainMenu`

Assets And Content

Environment

- `Assets/DungeonModularPack` provides the modular dungeon geometry and materials used by the maze builder
- Runtime maze geometry is assembled from assigned floor and wall prefabs rather than one authored level mesh

Character Art

- `Assets/Art/Mixamo` and `Assets/Art/Animations` contain player and ghost animation assets and animator controllers
- `PlayerAnimator.controller` and `CreepAnimator.controller` are the main animation state machines to verify if characters stop animating correctly

Audio

- `Assets/Audio/Ambient/Ambient ruins.mp3`
- `Assets/Audio/Music/Chess_duel.mp3`
- `Assets/Audio/SFX/Ghost_Alert.mp3`
- `Assets/Audio/SFX/Ghost_Roar.mp3`
- `Assets/Audio/SFX/FootSteps_Run.wav`

These clips support the tension curve across exploration, detection, and duel states.

UI

- `Assets/UI` contains HUD-facing art
- `Assets/TextMesh Pro` contains TMPro dependencies and examples
- `Assets/UI/Sprites/Chess_Pieces` supplies the duel piece icons

Scene Setup Expectations

For a gameplay scene to function correctly, it should include:

- A persistent managers object with `GameStateManager`, `HealthSystem`, `PieceCollectionSystem`, `NoiseSystem`, and `ChessDuelManager`
- A maze root with `MazeGenerator`, `NavMeshSurface`, and optionally `MinimapFog`
- A player object with `CharacterController`, `PlayerInput`, `PlayerController`, animator, and footstep audio source
- A ghost object with `NavMeshAgent`, `GhostAI`, and `GhostAnimator`
- A UI canvas with `UIManager`, `HUDController`, `ChessBoardUI`, and end-screen buttons
- A minimap camera using `MinimapController`

Production Notes

Debug / QA Notes

Current script-level debug shortcuts in `PlayerController`:

- `P` forces the game into chess duel state
- `L` removes one heart

These are useful for testing, but should be removed or gated before a production release build.

Known Architectural Notes

- `GameStateManager` is marked `DontDestroyOnLoad`, so duplicate manager objects across scenes must be avoided
- `MainMenuManager` stores scene names as strings, so scene renames must be reflected in the Inspector
- `PlayerSpawner` and `GhostSpawner` appear to be legacy or expansion scripts, while the active procedural flow is driven by `MazeGenerator`
- `ChessBoardUI` declares a `lastMoveColor` field that is not currently applied in the board refresh path

Credits

Course: `CMPS434`

Team:

- Majid Marzouq | Gameplay Programmer
- Oluwadamilola Olajide | Level Designer
- Abdullah Hamed | UI Artist
- Mohamad Allouh | Narrative Designer

Supporting docs:

- `docs/GAME_DESIGN.md`
- `docs/SCRIPTS_REFERENCE.md`
- `cmps434/course_content/lectures`